

Project Master Document

Sebastian Hönel

Last Update: September 1, 2026

Contents

1 About	1
2 Suggested milestones	2
2.1 First Tutoring Session – Project Proposal Presentation	2
2.2 After Four Lectures – Intermediate Presentation	2
2.3 Final submission	3
3 Common project constraints	4
3.1 Common interpretation of grading criteria	4
3.2 Common requirements for all four selected themes	4
4 Per-theme instructions	5
4.1 Data Handling	5
4.2 Outlier Detection	5
4.3 Association Rule Mining	6
4.4 Clustering	7
4.5 Dimensionality Reduction	8
4.6 Networks	9
4.7 Text Mining	10
5 Some Ideas – food for thought	11
5.1 Detecting and explaining unusual product reviews	11
5.2 Mapping discussion communities and emerging topics on Reddit	11
5.3 Market-basket patterns and unusual purchasing behaviour	12
5.4 Understanding cyclist or traffic demand patterns	13
5.5 Exploring research trends and collaboration structures	13
5.6 Classifying and relating songs through lyrics and audio metadata	14

1. About

This is your master document for project-related work in 4DV510 / 4DT910. It includes suggestions for how to handle the three milestones, common project constraints, grading criteria, and per-theme instructions. Every proposal must use exactly **four** lecture themes and can be scoped to approximately 84 person-hours per group (3 students x 4 hours x 7 weeks). For groups of two, this reduces to 56 person-hours over the span of the course. Projects shall use Python exclusively (there may be exceptions).

You will also find six deliberately broad project directions. They are intended as inspiration rather than fixed assignments: each can be narrowed to a concrete research question, dataset, and evaluation plan during the first tutoring session.

The projects should emphasize an end-to-end data-mining pipeline: data acquisition/handling, justified methodological choices, data-quality discussion, implementation, and evaluation. This aligns with the

course objectives on designing data-mining pipelines, implementing algorithms, extracting data from unstructured sources, and reflecting on data quality and societal implications.

2. Suggested milestones

2.1. First Tutotoring Session – Project Proposal Presentation

- Project aim and motivation:
 - Defined several objectives or research questions
 - Motivation why the analysis is useful
- Data:
 - Dataset source with brief description (variables, data types, ground truth, size, volume etc.); expected data volume
 - Licence/access conditions
- Selection of data mining topics:
 - Select a minimum of **four** data mining topics (e.g., clustering, outlier detection, association rule mining, etc.)
 - Explain how each topic contributes to one or more analysis objectives or research questions
- Initial pipeline diagram:
 - Provide a simple sketch or diagram showing the planned workflow
 - Show which data mining techniques will be used for which objective / question and how they are connected
- Expected evaluation approach:
 - For supervised methods: how will performance be evaluated against the ground truth?
 - For unsupervised methods: how will the quality, stability, usefulness, or interpretability of the results be assessed?
- Risks, including unavailable data or excessive scope

Note: You must submit your proposal, having incorporated any feedback you got from us during the presentation on **the same day**. You will get feedback from us latest the next day. You **must not start working** before that.

2.2. After Four Lectures – Intermediate Presentation

- Data exploration
 - Dataset description: source, size, variables/features, and data types
 - Descriptive statistics and appropriate/relevant visualizations
 - Important data-quality findings
- Feature/variable selection
 - Which variables/features were selected?
 - Why are they relevant to the analysis objectives/questions?
- Data preprocessing and cleaning
 - Data-cleaning steps performed
 - Handling of missing values, duplicates, incorrect/unusual values, and outliers where relevant
 - Transformations, encoding, normalization, or text preprocessing where relevant
- Completed analysis of **two** selected data mining topics
 - Complete implementation of methods/algorithms for both topics
 - Present and interpret the results
 - Evaluate the results using appropriate evaluation methods
 - Answer the corresponding analysis objectives/questions
 - Discuss limitations, problems, and unsuccessful approaches
- Concrete plan for the remaining **two** data mining topics
 - Which two topics will be implemented next?
 - Which analysis objective/question will each address?

- Which methods/algorithms are planned?
- What evaluation strategy will be used?
- Updates to the original project plan
 - Any changes to the objectives/questions, methods, data, or scope
 - Brief explanation of why the changes were needed

2.3. Final submission

Deliverables:

- Source code and reproducible instructions.
- Dataset acquisition/preparation instructions, without uploading data that cannot legally be redistributed (share that only privately with the teachers).
- Presentation and Group Project Report (see below).

Final presentation

- 10 minutes + 3-5 minutes questions and discussion
- Motivation and project (research) question(s).
- Data: Overview, descriptive table, network summary, etc.
- Pipeline(-diagram) and selected themes.
- Methods and experimental design.
- Main results: Also include one evaluation or comparison result.
- Critical interpretation, limitations, and conclusions.

Group Project Report

- Ca. 2 pages, excluding references and optional appendix.
- What did the group build, analyze, and find? How can it be reproduced?
- Document the project in a compact form: what was investigated, what data and methods were used, what was found, how can it be reproduced, etc.
- Also report on methods, results, limitations, data quality, ethics, group process, and what would be improved with more time.
- Focus on the project artifacts and results, not individual learning processes, division of labor, or extended reflections.
- Give an overall impression about your experience (positive, negative) with using the language, tools, frameworks, and algorithms that were used. What could have been better, and what were the obstacles you found in the way.
- Describe an example of a pipeline that includes data loading, mining (clustering, dimensionality reduction, community detection, etc.) and present the results. Include the code as figure or a specially-formatted text section. Show how your framework allows for different techniques to be applied in each phase without changing the skeleton of the pipeline.
- Did the framework help you better understand the overarching concepts of data mining? Describe shortly what you learned about the structures of each general concept that was implemented, and how they all interoperate at a higher level.

3. Common project constraints

- **Exactly four themes:** each group explicitly identifies its four selected themes and explains how each is substantively used – not merely mentioned.
- **A clear question:** the project should seek to discover, compare, explain, or evaluate something specific; “apply several algorithms to a dataset” is not enough.
 - This is not a strict requirement. It might be worth for the initial proposal and your final project to define and address **up to three** questions.
- **A manageable dataset:** groups should select a limited time window, subset, language, city, category, or sample size where necessary.
- **Data-quality analysis:** students should document missingness, duplicates, bias, uncertain labels, sampling, preprocessing choices, and likely effects on conclusions. Keep in mind that one focus point of the course is data quality and its effect on extracted knowledge.
- **Evaluation:** where labels exist, use appropriate external evaluation. Otherwise use internal quality measures, stability checks, qualitative inspection, and domain-relevant interpretation.
- **Ethics and societal reflection:** particularly for user-generated content, behavioural data, health-related data, or potentially sensitive inferences.

3.1. Common interpretation of grading criteria

For each selected theme, we award one point for each of the four criteria below that is demonstrably fulfilled:

- **0 points:** No meaningful implementation or analysis of the theme.
- **1 point:** Basic implementation exists, but is incomplete, undocumented, or not connected to the project question.
- **2 points:** Minimum technical requirements are fulfilled.
- **3 points:** The group compares alternatives, justifies choices, and evaluates results.
- **4 points:** The group additionally provides a convincing interpretation of results, limitations, and implications for the overall project.

Alternatively, the four requirements under each theme (see below) may be used directly as a four-point checklist in order to make the assessment transparent. Every criterion should be supported by source code, output, figures/tables, and discussion in the report.

3.2. Common requirements for all four selected themes

In addition to the theme-specific requirements, each group should include the following in their final submission:

1. A clearly formulated project question.
2. A pipeline diagram showing how the selected four themes connect.
3. Reproducible source code and execution instructions.
4. A description of datasets, licences, provenance, and preprocessing.
5. A results section containing both quantitative output and substantive interpretation.
6. A limitations section addressing data quality, methodological uncertainty, and ethical or societal issues.

This is particularly important because the course learning outcomes require students not only to apply techniques, but also to reason about data quality and its consequences, and to reflect on ethical and value-related aspects of data mining.

4. Per-theme instructions

4.1. Data Handling

4.1.1. Introduction

Data handling concerns acquiring, representing, cleaning, transforming, integrating, and documenting data before analysis. Poor data quality can substantially affect patterns, models, and conclusions. This theme should therefore not be interpreted as a purely technical preprocessing task: students should also assess what data limitations mean for their findings. The course explicitly includes data quality and asks students to reason about how it affects extracted knowledge.

4.1.2. Minimum requirements

A project selecting Data Handling must:

1. **Acquire and document data**
 - Use an open dataset where possible, or clearly document the source, licence, collection method, access date, and inclusion/exclusion criteria.
 - Describe the unit of analysis: for example, a customer, transaction, document, author, network node, station-day, or review.
2. **Implement a reproducible ingestion and preparation pipeline**
 - Code must load the raw data and produce the analysis-ready dataset without manual spreadsheet edits.
 - Intermediate datasets may be saved, but their creation must be reproducible.
3. **Perform and justify data-quality operations**
 - Include at least three relevant operations, such as handling missing values, duplicates, inconsistent labels, invalid values, date parsing, text cleaning, aggregation, feature construction, joining sources, sampling, or normalization.
 - Each operation must be justified.
4. **Report data quality and limitations**
 - Provide descriptive statistics and visualisations appropriate to the data.
 - Discuss important limitations, such as missingness, measurement error, selection bias, temporal bias, dataset coverage, uncertain labels, or class imbalance.

4.1.3. Objective grading criteria

Criterion	Evidence expected
Data provenance and unit of analysis are clearly documented	Dataset/source description and reproducible acquisition instructions
A fully reproducible loading and preprocessing pipeline is supplied	Executable code or notebook from raw input to prepared data
At least three relevant data-quality/preprocessing operations are implemented and justified	Code plus before/after summaries or examples
Data-quality limitations and their effect on conclusions are analysed	Report section with descriptive results and critical discussion

4.2. Outlier Detection

4.2.1. Introduction

Outlier detection identifies observations that deviate substantially from expected patterns. Outliers may be errors, rare but valid events, novel behaviours, fraud-like cases, measurement anomalies, or indicators

of important subgroups. Students must avoid assuming that an outlier is automatically an error or malicious observation.

4.2.2. Minimum requirements

A project selecting Outlier Detection must:

1. **Define what an outlier means in the project context**
 - Specify whether anomalies are sought at the level of records, users, transactions, documents, time periods, nodes, or another unit.
 - Explain why unusual cases are relevant to the research question.
2. **Construct suitable features and preprocessing**
 - Describe the representation used for outlier detection.
 - Address scaling, skewed variables, sparsity, missing values, and categorical/text/network-derived features where relevant.
3. **Compare at least three detection approaches**
 - These may include statistical methods, distance-based methods, density-based methods, isolation-based methods, clustering-based methods, or domain-specific rule-based baselines.
 - At least one method should be a simple interpretable baseline.
4. **Evaluate and interpret detected outliers**
 - If labels exist, use suitable supervised evaluation measures.
 - If labels do not exist, use methods such as stability checks, overlap between methods, manual inspection, domain rules, or representative-case analysis.
 - Explain likely false positives and false negatives.

4.2.3. Objective grading criteria

Criterion	Evidence expected
The anomaly definition, observation unit, and features are explicit and justified	Clear problem definition and feature description
At least three substantively different detection approaches are implemented	Code and parameter settings
Methods are compared using an appropriate evaluation strategy	Metrics, overlap/stability analysis, or systematic inspection
Detected cases are interpreted critically rather than merely listed	Examples, visualisations, and discussion of uncertainty/false positives

4.3. Association Rule Mining

4.3.1. Introduction

Association rule mining discovers co-occurrence patterns, typically in transaction or event data. Rules should not be judged only by confidence: high confidence can arise from very common items. Students should use appropriate interestingness measures and distinguish statistically frequent patterns from practically meaningful ones.

4.3.2. Minimum requirements

A project selecting Association Rule Mining must:

1. **Define transactions, items, and the analysis scope**
 - Explain what constitutes a basket/transaction and an item.
 - Document filtering decisions, such as minimum item frequency, time window, product category selection, or user subset.

2. Prepare data for rule mining

- Transform raw data into transaction-item form.
- Address duplicate items, cancellations/returns where relevant, rare items, missing identifiers, and excessively sparse data.

3. Mine and compare rule sets

- Use at least three alternative mining configurations or approaches.
- Alternatives may differ by algorithm, support threshold, confidence threshold, item representation, transaction definition, or constraint type.
- Report support, confidence, lift, and at least one additional relevant measure where appropriate.

4. Filter, evaluate, and interpret rules

- Apply principled filtering to avoid redundant, trivial, or spurious rules.
- Present a limited number of meaningful rules with contextual interpretation.
- Where possible, validate rules on a held-out time period or data subset.

4.3.3. Objective grading criteria

Criterion	Evidence expected
Transactions and items are well defined and reproducibly constructed	Code and description of transaction generation
At least three rule-mining configurations or approaches are compared	Parameter table and generated rule sets
Appropriate interestingness measures and filtering are used	Support/confidence/lift and rationale for rule selection
Final rules are evaluated and interpreted in the domain context	Held-out validation, examples, and discussion of practical relevance

4.4. Clustering

4.4.1. Introduction

Clustering identifies cohesive groups of similar observations. Most clustering methods operate on structured data, may require hyper-parameters, and return a cluster assignment or cluster description. Distance measures and clustering-quality measures are central parts of the analysis.

4.4.2. Minimum requirements

The existing clustering specification can be used directly. A project selecting Clustering must implement components for:

1. Dataset

- Exposes data points/rows and features/columns.

2. Distance Measure

- Takes two data points as input and returns one distance value.

3. Clustering Technique

- Takes a dataset and optional hyper-parameters as input.
- Hyper-parameters must not be hard-coded.
- A distance measure should be supplied as a parameter where applicable.

4. Clustering Quality Measure

- Takes a clustering result and a dataset as input and returns a single quality value.

The clustering specification requires at least three implementations for each relevant component and a programmatically executed evaluation pipeline. The pipeline should test three clustering techniques, three datasets, and three clustering quality measures, producing at least 27 runs, excluding optional hyper-parameter searches [2].

4.4.3. Objective grading criteria

Criterion	Evidence expected
Dataset, distance-measure, clustering, and quality-measure components are implemented with suitable interfaces	Source code and brief architecture documentation
At least three implementations of the required components are available	Code and list of supported techniques/measures
The evaluation pipeline executes at least 27 required combinations reproducibly	Automatically generated results table/log
Cluster quality, parameter sensitivity, and cluster meaning are interpreted critically	Quality-measure comparison, visualisation where suitable, and discussion

For this theme, satisfying only the minimum component requirements should normally correspond to a passing level. Higher achievement requires a well-designed experimental comparison and a thoughtful interpretation of whether the discovered clusters are meaningful.

4.5. Dimensionality Reduction

4.5.1. Introduction

Dimensionality reduction transforms high-dimensional data into a lower-dimensional representation. It can support visualisation, noise reduction, computational efficiency, and subsequent data-mining tasks. However, low-dimensional visualisations can be misleading: apparent separations or clusters in a two-dimensional plot do not by themselves establish real structure in the original data.

4.5.2. Minimum requirements

A project selecting Dimensionality Reduction must:

- 1. Define the high-dimensional representation**
 - Explain what the original features represent and why dimensionality reduction is appropriate.
 - Address scaling, sparsity, feature selection, and missing values as relevant.
- 2. Apply at least three dimensionality-reduction approaches or configurations**
 - Examples include PCA, truncated SVD, MDS, t-SNE, UMAP, autoencoder-based reduction, or self-organizing maps.
 - Merely varying the random seed is not sufficient; the alternatives must be substantively different methods or parameterisations.
- 3. Evaluate information preservation and/or downstream usefulness**
 - Use appropriate criteria, such as explained variance, reconstruction error, distance/neighbourhood preservation, trustworthiness, stability, visual separation relative to known labels, or performance in a downstream task.
- 4. Provide careful visual and substantive interpretation**
 - Use plots with meaningful colouring, annotations, or representative examples.
 - Discuss the effect of hyper-parameters and the risk of over-interpreting a visualisation.

4.5.3. Objective grading criteria

Criterion	Evidence expected
The original feature space and preprocessing choices are justified	Feature description and preprocessing code

Criterion	Evidence expected
At least three meaningful reduction methods/configurations are implemented	Code and parameter table
Reduction quality or downstream utility is evaluated quantitatively where possible	Metrics, comparison table, or downstream-task results
Visualisations are interpreted critically and connected to the research question	Annotated plots and discussion of limitations

4.6. Networks

4.6.1. Introduction

Network analysis studies entities and their relationships. Nodes may represent people, organisations, documents, products, web pages, or locations; edges may represent communication, collaboration, citations, hyperlinks, similarity, transactions, or co-occurrence. Network construction is itself a modelling decision and must be documented carefully.

4.6.2. Minimum requirements

A project selecting Networks must:

- 1. Construct and document a network**
 - Define nodes, edges, edge direction, edge weights, time period, and filtering criteria.
 - Justify why the constructed network represents a meaningful relationship.
- 2. Analyse network structure**
 - Calculate basic descriptive network statistics, such as number of nodes and edges, density, degree distribution, connected components, reciprocity, clustering coefficient, or path-length-related measures as appropriate.
- 3. Use at least three network-analysis methods**
 - Suitable alternatives include degree centrality, betweenness centrality, closeness centrality, eigenvector centrality, PageRank, HITS, community detection, link prediction, motif analysis, or temporal network analysis.
 - The methods should be appropriate to whether the graph is directed, weighted, bipartite, or temporal.
- 4. Interpret network findings in relation to the data-generating process**
 - Investigate central nodes, communities, bridges, isolated components, or changes over time.
 - Discuss potential biases introduced by missing nodes/edges, platform-specific behaviour, or network-construction decisions.

4.6.3. Objective grading criteria

Criterion	Evidence expected
Network nodes, edges, direction/weights, and construction rules are clearly defined	Network-construction code and documentation
Basic network structure is described quantitatively	Table/figures of descriptive statistics
At least three suitable network-analysis methods are used and compared	Results for centrality/community/link-analysis methods
Findings are interpreted with attention to network-construction and sampling limitations	Network visualisations, examples, and critical discussion

4.7. Text Mining

4.7.1. Introduction

Text mining extracts structure, patterns, and information from unstructured or semi-structured text. Typical tasks include document representation, keyword extraction, topic discovery, sentiment analysis, similarity analysis, classification, and information extraction. The central challenge is to convert raw text into a representation suitable for the project's research question while retaining enough context for interpretation.

4.7.2. Minimum requirements

A project selecting Text Mining must:

1. Document the text source and corpus

- Specify the origin of texts, document type, language(s), time period, inclusion criteria, and corpus size.
- Respect licences, terms of use, privacy, and copyright restrictions.

2. Implement a transparent preprocessing pipeline

- Include appropriate operations such as encoding handling, tokenisation, lowercasing, stop-word handling, stemming/lemmatisation, URL/user-name removal, language filtering, duplicate detection, or treatment of emojis and punctuation.
- Justify important choices; preprocessing should not be treated as universally correct.

3. Compare at least three text representations or extraction approaches

- Examples include bag-of-words, TF-IDF, n-grams, word/sentence embeddings, topic modelling, named-entity extraction, sentiment features, keyword extraction, or lexicon-based features.
- The approaches must be connected to the project question.

4. Evaluate and interpret results

- Use labelled evaluation where labels exist; otherwise use topic coherence, stability, manual validation, error analysis, representative examples, or comparison to external metadata.
- Show examples of extracted text patterns, terms, topics, entities, or similar documents.

4.7.3. Objective grading criteria

Criterion	Evidence expected
Corpus provenance, scope, and legal/ethical considerations are documented	Dataset description and sourcing information
A reproducible and justified text-preprocessing pipeline is implemented	Code plus examples of preprocessing output
At least three text representations or extraction approaches are compared	Feature/method comparison and parameter documentation
Results are evaluated and interpreted using concrete textual examples	Metrics where possible, qualitative inspection, and error/limitation analysis

5. Some Ideas – food for thought

Here you will find some ideas. The idea is for you to take this as inspiration, not copy them 1:1.

5.1. Detecting and explaining unusual product reviews

Possible question:

Can suspicious or unusually influential reviews be identified, and do they form recognizable groups or patterns in review text and reviewer behaviour?

Themes (4):

1. Data Handling
2. Outlier Detection
3. Clustering
4. Text Mining

Possible open data:

- Amazon Reviews datasets
- Yelp Open Dataset
- Steam game reviews dataset (where available under suitable terms)

Possible workflow:

- Extract review text, ratings, timestamps, helpfulness votes, product/category metadata, and reviewer-level aggregates.
- Clean duplicate, missing, malformed, and very short reviews; discuss sampling bias and the representativeness of “suspicious” behaviour.
- Build textual representations such as TF-IDF vectors, selected sentiment/lexicon features, review length, rating deviation, posting frequency, and vocabulary diversity.
- Apply outlier detection to identify anomalous reviews or reviewers.
- Cluster reviews/reviewers to distinguish types of unusual activity – for example, generic repeated praise, bursts of negative reviews, extreme-rating reviewers, or unusually similar texts.
- Manually inspect a sample of detected cases and explain what the methods actually detect, rather than claiming that all outliers are fraudulent.

Good final output: A ranked and interpretable set of anomalous-review patterns, supported by cluster summaries, representative texts, and a careful discussion of false positives and ethics.

5.2. Mapping discussion communities and emerging topics on Reddit

Possible question:

How do communities form around a topic, and which topics or users connect otherwise separated discussion groups?

Themes (4):

1. Data Handling
2. Networks
3. Text Mining
4. Dimensionality Reduction

Possible open data:

- Public Reddit comment/submission datasets or carefully collected public subreddit data
- Pushshift-derived datasets, where legally and technically appropriate
- Alternative: public discussion datasets from Stack Exchange

Possible workflow:

- Select a limited, clearly defined topic and time period, such as climate policy, programming languages, gaming, or public health.
- Build a network of users, posts, or subreddits. Edges could represent replies, mentions, co-commenting, or shared vocabulary.
- Extract text features or topics from posts/comments.
- Use network measures such as degree, PageRank, centrality, or community detection to identify prominent users and communities. Link-analysis topics such as PageRank and HITS are explicitly part of the course content.
- Use dimensionality reduction to visualize documents, users, or communities from text-derived features.
- Compare network communities with text-based topic groupings: do socially connected users actually discuss similar topics?

Good final output:

An interactive or static map of the discussion network, a 2D topic visualization, and an analysis of bridge users/communities and their thematic role.

Important limitation:

Students should avoid collecting personal data unnecessarily and should anonymize identifiers in reports and presentations.

5.3. Market-basket patterns and unusual purchasing behaviour

Possible question:

What products are commonly purchased together, which customer baskets are unusual, and can basket types be summarized into useful segments?

Themes (4):

1. Data Handling
2. Association Rule Mining
3. Outlier Detection
4. Clustering

Possible open data:

- UCI Online Retail dataset
- Instacart Market Basket Analysis dataset
- Groceries dataset or other public transactional retail datasets

Possible workflow:

- Transform transaction rows into baskets; handle cancellations, returns, duplicate records, missing product descriptions, and varying basket sizes.
- Mine association rules, comparing support, confidence, lift, and possibly conviction.
- Identify unusual baskets using features such as number of items, total quantity, rarity of products, rule violations, and time of purchase.

- Cluster baskets or customers based on product categories, item diversity, basket size, and/or rule-derived features.
- Compare whether anomalous baskets belong to particular customer or product clusters.

Good final output:

A compact set of actionable and non-trivial product associations, customer/basket segments, and a documented analysis of unusual transactions.

Potential methodological challenge:

Association rules can produce huge numbers of weak or redundant rules. A good project should define principled filtering and explain why selected rules are meaningful.

5.4. Understanding cyclist or traffic demand patterns

Possible question:

Can patterns in urban mobility demand be discovered, and which observations correspond to unusual weather, events, or operational conditions?

Themes (4):

1. Data Handling
2. Outlier Detection
3. Clustering
4. Dimensionality Reduction

Possible open data:

- City bike-sharing trip data, for example from Chicago, New York City, or London
- Public traffic-count datasets from municipalities
- Open weather data, optionally joined by date/time

Possible workflow:

- Aggregate trips into meaningful units: station-day, station-hour, route-day, or city-day.
- Engineer interpretable features: trip count, duration distribution, start/end imbalance, week-day/weekend, hour, weather, and station location.
- Detect outlying days or stations – for example, exceptionally high demand, unusually long trips, or severe imbalances.
- Cluster stations or time periods to discover commuting, leisure, tourist, and mixed-use patterns.
- Use dimensionality reduction to visualize stations/days and assess whether clusters and anomalies are separated coherently.

Good final output:

A visual typology of stations or mobility periods, together with explanations for selected anomalies. Students should distinguish operationally interesting anomalies from mere data errors.

5.5. Exploring research trends and collaboration structures

Possible question:

How have research topics evolved in a selected field, and how do topic similarity and collaboration networks relate?

Themes (4):

1. Data Handling

2. Text Mining
3. Networks
4. Clustering

Possible open data:

- OpenAlex data/API
- Crossref metadata
- arXiv metadata and abstracts
- PubMed Open Access subsets, subject to the selected scope

Possible workflow:

- Select a manageable discipline and publication period – for example, machine learning, cybersecurity, bioinformatics, or digital humanities.
- Obtain titles, abstracts, author lists, venues, keywords, citations, and years where available.
- Preprocess titles and abstracts; create document features and identify prominent terms/topics.
- Build an author collaboration network or a citation network.
- Cluster publications based on textual similarity, and compare these clusters to network communities, venues, or time periods.
- Identify central authors/papers and investigate whether they bridge topical clusters.

Good final output:

A small “map” of a research field, showing topic clusters, collaboration/citation structures, and temporal trends.

Caution:

Author-name disambiguation is difficult. Students can acknowledge this as a data-quality limitation rather than trying to solve it completely.

5.6. Classifying and relating songs through lyrics and audio metadata

Possible question:

Do songs that are similar in lyrical content also form similar groups according to genre, popularity, or audio characteristics?

Themes (4):

1. Data Handling
2. Text Mining
3. Clustering
4. Dimensionality Reduction

Possible open data:

- Million Song Dataset metadata
- Free Music Archive metadata
- Public song lyric datasets with clear reuse permissions
- Spotify-derived feature datasets, provided terms and provenance are respected

Possible workflow:

- Join lyric text, artist/genre metadata, release year, and available audio descriptors.
- Clean language, duplicated songs, missing lyrics, and inconsistent genre labels.
- Represent lyrics using TF-IDF, n-grams, sentiment/emotion features, or selected topic features.

- Cluster songs using lyric features, audio features, or a combination, and compare the results.
- Use dimensionality reduction to show whether songs separate by genre, period, popularity, or lyrical style.
- Interpret clusters through representative words, songs, artists, and metadata distributions.

Good final output:

A visual and interpretable comparison between lyrical similarity, genre labels, and audio/metadata-derived similarity.

Caution:

Genre is not an objective ground truth, and lyrics may be copyrighted even when metadata is open. The project should use permitted data and avoid redistributing copyrighted text.